

**MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION
TECHNOLOGY
FACULTY OF SOFTWARE ENGINEERING**



**SOFTWARE DEVELOPMENT THESIS PROJECT
COURSE: CT250H**

Thesis title

**DESIGN AND IMPLEMENTATION OF A HUMAN-IN-THE-LOOP
AI-ASSISTED RECRUITMENT AUTOMATION PLATFORM
FOR CV-JD EVALUATION AND RECOMMENDATION IN IT**

Student

Pham Huu Hung - B2203557

Can Tho, August 2026

**MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION
TECHNOLOGY
FACULTY OF SOFTWARE ENGINEERING**



**SOFTWARE DEVELOPMENT THESIS PROJECT
COURSE: CT250H**

Thesis title

**DESIGN AND IMPLEMENTATION OF A HUMAN-IN-THE-LOOP
AI-ASSISTED RECRUITMENT AUTOMATION PLATFORM
FOR CV-JD EVALUATION AND RECOMMENDATION IN IT**

**Supervisor
Ph.D. Nguyen Thanh Khoa**

**Student
Pham Huu Hung - B2203557**

Can Tho, August 2026

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to Ph.D. Nguyen Thanh Khoa for the guidance, constructive feedback, and continuous support provided throughout this thesis. The supervisor's academic and technical advice played an important role in shaping the research direction and improving the quality of this work.

I am also grateful to the lecturers and staff of the College of Information and Communication Technology, Can Tho University, for providing the academic foundation and learning environment necessary for the completion of this project. My thanks are extended to everyone who contributed feedback during the design, implementation, and evaluation of the CareerFit system.

Finally, I would like to thank my family and friends for their encouragement and support throughout my studies. Although every effort has been made to ensure the accuracy and quality of this thesis, limitations may remain, and constructive comments are sincerely appreciated.

Can Tho, August 2026
Pham Huu Hung

DECLARATION OF ORIGINALITY

I hereby declare that this thesis, titled “Design and Implementation of a Human-in-the-Loop AI-Assisted Recruitment Automation Platform for CV-JD Evaluation and Recommendation in IT,” is my original work completed under the supervision of Ph.D. Nguyen Thanh Khoa. The implementation, experiments, data analysis, and conclusions presented in this report are reported truthfully within the stated scope.

All ideas, publications, standards, software documentation, figures, and other materials obtained from external sources are acknowledged through citations or source notes. I accept full responsibility for the accuracy, academic integrity, and originality of the submitted work.

Can Tho, August 2026
Student

Pham Huu Hung

SUPERVISOR'S COMMENTS

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Can Tho, August 2026
Supervisor

Ph.D. Nguyen Thanh Khoa

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	i
DECLARATION OF ORIGINALITY	ii
SUPERVISOR'S COMMENTS	iii
TABLE OF CONTENTS.....	iv
LIST OF TABLES.....	vii
LIST OF FIGURES	viii
LIST OF ABBREVIATIONS	ix
ABSTRACT.....	x
INTRODUCTION.....	1
Problem Statement.....	1
Background and Related Work	2
Research Objectives	3
Overall Objective.....	3
Specific Objectives.....	3
Research Subjects and Scope	4
Research Subject.....	4
Research Scope.....	4
Research Methods and Content	5
Research Methods.....	5
Research Content.....	5
Key Contributions	5
Thesis Structure	6
MAIN CONTENT.....	7
CHAPTER 1. PROBLEM DESCRIPTION AND REQUIREMENTS	7
1.1 System Analysis Context	7
1.2 Stakeholders and Actors	8
1.3 Functional Requirements.....	9
1.4 Non-Functional Requirements.....	10
1.5 Use-Case Analysis	11
1.5.1 Candidate Uploads a CV and Receives Matches.....	11
1.5.2 Candidate Searches, Applies, and Withdraws.....	12
1.5.3 Recruiter Creates a Job and Reviews Candidates	12
1.5.4 Candidate Feedback Changes Later Ranking	13
1.5.5 AutoFit Executes a Policy-Governed Action.....	13
1.5.6 User Completes an Email Feedback Action	14
1.6 Chapter Summary	15
CHAPTER 2. THEORETICAL BACKGROUND AND SOLUTION DESIGN.....	16
2.1 Recruitment Information and CV–Job Description Matching.....	16
2.2 Text Representation and Preprocessing	17
2.2.1 Document Normalization.....	17
2.2.2 Bag-of-Words Representation.....	18
2.3 TF-IDF and Cosine Similarity	18
2.3.1 Term Frequency and Inverse Document Frequency	18
2.3.2 Cosine Similarity	18
2.4 Matching, Recommendation, and Ranking.....	19
2.5 Relevance Feedback and the Rocchio Method.....	20
2.5.1 Relevance Feedback	20
2.5.2 Feedback Semantics in CareerFit	21
2.6 Evaluation Metrics for Ranked Results	22
2.6.1 Precision@K, Recall@K, and HitRate@K	22
2.6.2 Mean Reciprocal Rank	22
2.6.3 Discounted Cumulative Gain.....	23

2.7 Human-in-the-Loop and Explainable Automation	23
2.7.1 Levels of Human Involvement	23
2.7.2 Explainability and Auditability	23
2.8 Web Architecture, Security, and Audit Foundations	25
2.8.1 Client–Server and REST	25
2.8.2 Authentication, Authorization, and Action Tokens	25
2.8.3 Audit Logging	26
2.9 Related Work and Research Gap	26
2.10 Theoretical Background Summary	27
2.11 System Architecture	27
2.12 Module Design	28
2.13 Data Design	29
2.13.1 Core Identity and Recruitment Data	29
2.13.2 Automation, Communication, and Operational Data	30
2.14 Security, Failure, and Consistency Design	31
2.14.1 Authentication and Authorization	31
2.14.2 Failure Handling	32
2.14.3 Identified Design Risks	32
2.15 Deployment Architecture	33
2.16 Chapter Summary	34
CHAPTER 3. SYSTEM IMPLEMENTATION	35
3.1 Implementation Overview	35
3.2 Authentication and Authorization Implementation	36
3.2.1 Login and JWT Processing	36
3.2.2 URL Rules and Ownership Checks	37
3.3 CV Ingestion and Validation	38
3.3.1 Upload and Manual Creation	38
3.3.2 File Validation and Extraction	38
3.4 Text Normalization and TF-IDF Implementation	39
3.4.1 Text Normalization	39
3.4.2 Static Corpus and Vector Construction	40
3.5 Matching and Recommendation Implementation	41
3.5.1 Scoring Service	41
3.5.2 Matching Orchestration and Persistence	41
3.5.3 Recommendation Service	42
3.6 Feedback Learning with Rocchio	42
3.7 Application and Recruiter Workflow	44
3.8 AutoFit and Background Processing	45
3.8.1 Async Executor and Scheduler	45
3.8.2 Auto-Apply	45
3.8.3 Notification Guard and Delivery	45
3.9 Email Action and Audit Implementation	46
3.10 Persistence and API Implementation	47
3.11 Frontend Integration	48
3.12 Deployment and Observability Implementation	52
3.13 Implementation Limitations	53
3.14 Chapter Summary	54
CHAPTER 4. TESTING AND EVALUATION	55
4.1 Evaluation Objectives	55
4.2 Evaluation Environment	55
4.3 Dataset and Evaluation Design	56
4.3.1 Controlled Algorithm Dataset	56
4.3.2 Local Runtime Data	57
4.4 Backend Automated Testing	57
4.5 Controlled Algorithm Results	58

4.6 Concurrency Observation in Benchmark Runs	59
4.7 Frontend Build Evaluation.....	59
4.8 Browser End-to-End Evaluation.....	60
4.9 Security-Oriented API Checks.....	61
4.10 Runtime Health, Monitoring, and Local Latency	62
4.11 Evaluation Summary by Research Question.....	63
4.12 Threats to Validity.....	63
4.12.1 Construct Validity	63
4.12.2 Internal Validity	64
4.12.3 External Validity.....	64
4.12.4 Reproducibility	64
4.13 Chapter Summary	64
CONCLUSION	65
1. Summary of Results.....	65
2. Achievement of Thesis Objectives	66
2.1 Role-Based Recruitment Platform.....	66
2.2 CV and Job-Description Processing	66
2.3 Matching and Recommendation	66
2.4 Feedback Learning	67
2.5 Policy-Driven Automation and Email Action	67
2.6 Auditability and Evaluation	67
3. Discussion.....	68
3.1 Value of an Interpretable Baseline	68
3.2 Human-in-the-Loop as System Structure	68
3.3 Meaning of the Rocchio Improvement	69
3.4 Test Results versus Background Errors	69
3.5 Product Positioning.....	69
4. Limitations.....	70
4.1 Data and Algorithm Limitations	70
4.2 Evaluation Limitations	70
4.3 Security and Privacy Limitations.....	70
4.4 Reliability and Maintainability Limitations.....	70
5. Future Work	71
6. Closing Remarks	71
REFERENCES	73
APPENDICES	75
Appendix A. Requirement-Test Traceability Matrix	75
Appendix B. Selected API Contracts.....	75
Appendix C. Data Model Summary	75
Appendix D. Evaluation Summary.....	75
Appendix E. UAT and Demonstration Script	76
Preparation.....	76
Execution.....	76
Acceptance and cleanup	76
Appendix F. Local Deployment Instructions.....	77
Prerequisites	77
Startup procedure.....	77
Verification	77
Shutdown and cleanup.....	77
Appendix G. Role-Based User Guide.....	78
Guest.....	78
Candidate.....	78
Recruiter	78
Administrator.....	78

LIST OF TABLES

Table 1.1. Mapping of objectives, contributions, and evidence.....	6
Table 1.2. Actors and responsibilities	8
Table 1.3. Consolidated functional requirements	9
Table 1.4. Non-functional requirements and design responses	10
Table 1.5. Use case - Candidate uploads a CV and receives matches	11
Table 1.6. Use case - Candidate searches, applies, and withdraws.....	12
Table 1.7. Use case - Recruiter creates a Job and reviews candidates.....	12
Table 1.8. Use case - Candidate feedback changes later ranking	13
Table 1.9. Use case - AutoFit executes a policy-governed action	13
Table 1.10. Use case - User completes an email feedback action.....	14
Table 2.1. Positioning of CareerFit against major approach categories	26
Table 2.2. Backend modules and responsibilities	28
Table 2.3. Key integrity constraints.....	30
Table 2.4. Design risks and required treatment	32
Table 3.1. Main implementation technologies.....	35
Table 3.2. CV processing states.....	38
Table 3.3. Implemented score interpretation	41
Table 3.4. Feedback handling	43
Table 3.5. Representative API-to-service mapping	47
Table 3.6. Verified implementation limitations	53
Table 4.1. Evaluation environment.....	55
Table 4.2. Backend automated test results.....	57
Table 4.3. Baseline and Rocchio results on the synthetic holdout scenario.....	58
Table 4.4. Frontend build artifacts.....	59
Table 4.5. Integrated Chrome workflow results	60
Table 4.6. API authorization observations.....	61
Table 4.7. Runtime observations	62
Table 4.8. Answers supported by current evidence	63
Table C.1. Consolidated result status.....	65
Table C.2. Objective assessment	67
Table C.3. Principal limitations and impact.....	70

LIST OF FIGURES

Figure 1.1. CareerFit problem context and principal information flows	2
Figure 1.2. Scope boundary of the CareerFit thesis.....	5
Figure 1.3. CareerFit system context.....	7
Figure 1.4. CareerFit use-case overview	9
Figure 1.5. CV ingestion and matching sequence	12
Figure 1.6. Feedback learning and recomputation sequence	13
Figure 1.7. AutoFit decision flow	14
Figure 1.8. Secure email-action confirmation and redemption sequence.....	15
Figure 2.1. Distinction between matching, recommendation, and recruitment action	17
Figure 2.2. TF-IDF vectorization and cosine-similarity pipeline	19
Figure 2.3. Rocchio relevance-feedback vector update.....	22
Figure 2.4. Human-in-the-Loop control cycle in CareerFit.....	24
Figure 2.5. CareerFit container and component architecture.....	28
Figure 2.6. CareerFit logical entity relationships	31
Figure 2.7. Local and containerized deployment topology	33
Figure 3.1. Backend module structure and request flow	36
Figure 3.2. JWT authentication and authorization boundaries	37
Figure 3.3. CV ingestion implementation pipeline.....	39
Figure 3.4. Seed-corpus initialization and TF-IDF construction	40
Figure 3.5. Scoring and matching persistence flow.....	42
Figure 3.6. Feedback processing and post-commit learning	43
Figure 3.7. Application and invitation state transitions	44
Figure 3.8. Scheduler and AutoFit execution boundaries.....	46
Figure 3.9. Hashed email-action token and confirm-then-POST flow	47
Figure 3.10. Frontend routes and API data flow.....	49
Screen 3.1. Candidate urgent-job catalogue	50
Screen 3.2. Candidate AutoFit policy settings.....	50
Screen 3.3. Candidate CV upload entry point	51
Screen 3.4. Recruiter Jobs and applicant workspace.....	51
Screen 3.5. Recruiter Talent Pool and Potential CV	52
Screen 3.6. Administrator audit logs	52
Figure 4.1. Evaluation environments and evidence sources.....	56
Figure 4.2. Baseline and Rocchio benchmark metrics at $K = 5$	59
Figure 4.3. P0 end-to-end workflow coverage	61
Figure 4.4. Local Job-search latency distribution.....	63

LIST OF ABBREVIATIONS

Abbreviation	Full Term	Meaning / Explanation
AI	Artificial Intelligence	Computer systems designed to perform tasks commonly associated with human intelligence
API	Application Programming Interface	A defined interface through which software components communicate
CV	Curriculum Vitae	A document describing a candidate's qualifications and experience
E2E	End-to-End	Testing of a complete workflow across integrated components
HITL	Human-in-the-Loop	Automation in which humans retain oversight or approval authority
JD	Job Description	A description of a job's responsibilities and requirements
JWT	JSON Web Token	A compact token format used for authentication and authorization
MRR	Mean Reciprocal Rank	A ranking metric based on the position of the first relevant item
nDCG	Normalized Discounted Cumulative Gain	A ranking metric that accounts for relevance and position
NLP	Natural Language Processing	Methods for processing and analyzing human language
OCR	Optical Character Recognition	Extraction of machine-readable text from images or scanned documents
RBAC	Role-Based Access Control	Authorization based on assigned user roles
REST	Representational State Transfer	An architectural style for networked APIs
TF-IDF	Term Frequency–Inverse Document Frequency	A weighting method for representing text terms
UAT	User Acceptance Testing	Validation of whether the system satisfies intended user workflows

ABSTRACT

Background: Recruitment in the information technology sector requires candidates and recruiters to compare many types of information from curricula vitae and job descriptions. Most job portals support posting, searching, and manual applications, while automated screening tools may not clearly explain how a score is calculated or how an automated action is controlled.

Objectives: This thesis designs and implements CareerFit IT AutoPilot, a web-based recruitment platform that combines a job portal, CV-job description matching, personalized job recommendations, policy-based automation, and Human-in-the-Loop interaction in one auditable workflow.

Methods: Recruitment text is normalized and represented with Term Frequency-Inverse Document Frequency vectors. Cosine similarity ranks CV-job pairs and candidate-job recommendations, while Rocchio relevance feedback updates learned representations from explicit feedback. A separate, versioned skill-transfer model checks aliases, skill families, shared foundations, and seniority to explain Potential candidates without changing the direct cosine score. AutoFit checks consent, thresholds, interaction history, quotas, cooldowns, time zones, and quiet hours before choosing an action. CareerFit uses a Spring Boot backend, a React and TypeScript frontend, and PostgreSQL persistence managed through Flyway migrations.

Results: On August 3, 2026, the refreshed backend suite passed 131 of 131 tests across 33 suites. TypeScript checking, ESLint, the production frontend build, and the bundle check also passed. All 41 Chrome workflow, contract, and resilience tests passed against the integrated application. In the controlled synthetic benchmark, $nDCG@5$ increased from 0.037737 to 0.837737 after Rocchio feedback. These results support a well-tested academic prototype, but they do not prove production readiness or real-world hiring effectiveness.

Keywords: *recruitment automation; CV-job matching; recommendation system; Human-in-the-Loop; Rocchio feedback; explainable automation.*

INTRODUCTION

Problem Statement

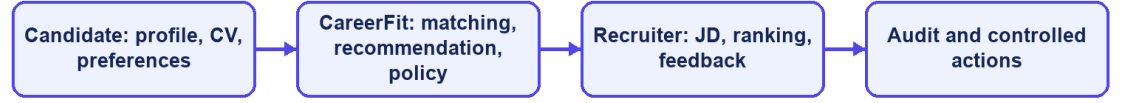
IT recruitment requires candidates and recruiters to compare many details, including technical skills, experience, language, location, and working conditions. Candidates must identify suitable vacancies, while recruiters must review CVs and decide which applicants deserve closer attention. As the number of jobs and profiles grows, doing this manually becomes slow and inconsistent [10].

Most job portals support posting, searching, and applying, but they do not fully explain why a job fits a candidate or why one applicant ranks above another. CareerFit therefore treats CV–JD matching and personalized recommendation as related but separate tasks. The first compares a specific CV and job; the second prioritizes jobs from a candidate's broader profile and preferences.

Automation can reduce repetitive work, but a similarity score should not directly trigger an important action. User consent, application state, previous interactions, thresholds, quotas, and special conditions also matter. CareerFit uses a Human-in-the-Loop design so users can review important actions and administrators can trace what happened.

This thesis develops CareerFit IT AutoPilot, an IT-focused web platform that combines job discovery, reviewed CV and JD processing, matching, recommendation, Rocchio feedback, AutoFit policies, secure email actions, recruiter talent discovery, and audit records. The system helps users make decisions but does not replace recruitment judgment.

CareerFit problem context



CareerFit IT AutoPilot — thesis diagram

Figure 1.1. CareerFit problem context and principal information flows

Background and Related Work

The first motivation is to reduce fragmentation. Candidates often move between job search, CV management, application history, and email, while recruiters switch between vacancy management, applicant review, candidate discovery, and reporting. CareerFit brings these activities into one role-based system and uses email as an additional action channel.

The second motivation is explainability. TF-IDF and cosine similarity do not capture meaning as deeply as modern embedding models, but their terms and weights can be inspected. This makes them suitable for an academic baseline in which the scoring process, limitations, and effect of feedback must be demonstrated clearly.

The third motivation is controlled adaptation and automation. AutoFit places policy checks between a score and an action, considering consent, thresholds, previous interactions, cooldown, quota, time zone, and quiet hours. Rocchio feedback then provides a transparent way to adjust later rankings from explicit positive and negative judgments without overwriting the original representation.

Related work includes classical lexical retrieval, learned resume-job representations, explainable recommendation, and human-centered governance. Lexical methods provide inspectable weights and matching reasons, while recent dense approaches improve semantic representation but require stronger data and evaluation assumptions [6]-[8]. Research on algorithmic hiring and explainable recommendation also shows that model performance, fairness, user control, and

explanation quality must be evaluated separately [9], [10], [14]. CareerFit addresses a narrower integration gap by combining portal workflows, explainable scoring, feedback learning, policy-controlled actions, and audit records in one academic prototype.

Research Objectives

Overall Objective

The overall objective of this thesis is to design, implement, and evaluate a web-based Human-in-the-Loop recruitment automation platform for the IT domain. The platform is intended to support CV–job description evaluation, personalized job recommendation, feedback-based adaptation, and policy-driven actions while preserving user control, explainability, security, and auditability.

Specific Objectives

To achieve the overall objective, the thesis defines the following specific objectives:

- Design a role-based web platform for guests, candidates, recruiters, and administrators, with appropriate public and authenticated workflows.
- Provide candidate profile and CV management, including upload, manual drafts, OCR-assisted extraction, section review, user correction, confirmation, validation, and quality feedback before matching.
- Implement separate CV–JD matching and candidate-profile recommendation workflows using a shared text-normalization, TF-IDF, and cosine-similarity pipeline.
- Present normalized scores, relevance labels, matching reasons, validation signals, and a separate Potential explanation based on transferable skills, skill families, shared foundations, and seniority compatibility.
- Integrate explicit user feedback and apply the Rocchio relevance-feedback method to update learned representations and recompute rankings reproducibly.
- Implement AutoFit policies that convert matching results into controlled outcomes according to consent, thresholds, interaction state, quota, cooldown, time-zone, and quiet-hour rules.
- Support expiring email-action links with hashed token storage, a non-mutating confirmation page, POST execution, action-state tracking, and audit logging.
- Evaluate the algorithmic behavior and system workflows using reproducible experiments, automated tests, scripted acceptance scenarios, and security and reliability checks appropriate to the project scope.

Research Subjects and Scope

Research Subject

The research subject is the IT recruitment workflow supported by a web platform, including CV and Job-description text, lexical ranking and recommendation, explicit feedback signals, policy-controlled automation, and auditable user and system actions.

Research Scope

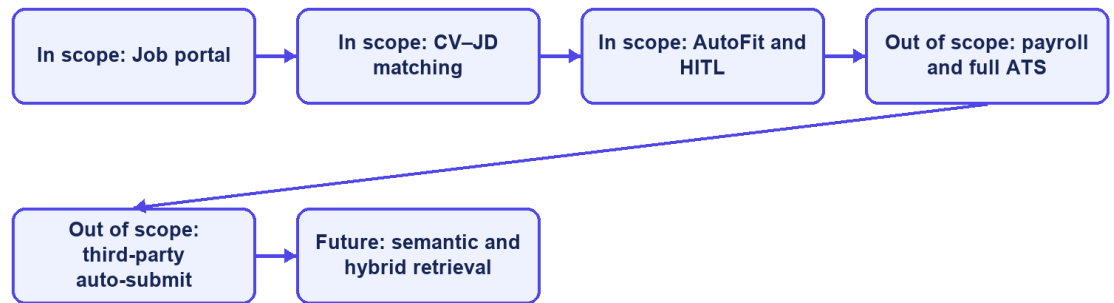
This thesis focuses on an academic prototype for IT recruitment. Its functional scope includes a public job portal; authenticated Candidate, Recruiter, and Administrator workspaces; reviewed CV and profile management; recruiter company profiles; Job drafts, quality preview, publishing, urgent and deadline fields; CV–JD matching; personalized recommendation; applications; Talent Pool bookmarks and invitations; explicit feedback; AutoFit policies; notifications; analytics; and audit records.

The data-processing scope includes extraction from supported CV documents, image preprocessing and OCR fallback, OCR cleanup, section review, bilingual-oriented text normalization, TF-IDF vectorization, cosine-similarity scoring, a versioned skill-transfer Potential assessment, relevance labeling, and Rocchio feedback updates. PostgreSQL is the main data store, and Flyway manages database changes. The React and TypeScript frontend calls REST APIs provided by the Spring Boot backend.

The thesis does not attempt to implement a complete enterprise applicant tracking system. Interview scheduling, offer management, payroll, and full employee lifecycle management are outside the core scope. The system does not automatically submit applications to external third-party job boards, and it does not use a large language model to make unrestricted recruitment decisions. Distributed microservices, large-scale message brokers, and semantic embedding models may be discussed as future extensions but must not be presented as implemented core components unless they are added and verified before the final report is submitted.

The evaluation scope is limited. Controlled or synthetic datasets can show whether Rocchio changes rankings in the expected direction, but they cannot prove recruitment quality in a real production setting. The source and purpose of scraped, seeded, or synthetic data must be stated. Claims about usability, performance, security, or production readiness are limited to the test environment, participants, procedures, and evidence available during the final evaluation.

Thesis scope boundary



CareerFit IT AutoPilot — thesis diagram

Figure 1.2. Scope boundary of the CareerFit thesis

Research Methods and Content

Research Methods

This thesis follows an applied software-engineering research approach. The work combines requirement analysis, architecture and data design, incremental implementation, a controlled algorithm experiment, automated backend and browser testing, runtime observation, and threats-to-validity analysis. Implementation claims are checked against source code, migrations, API contracts, logs, test reports, and generated evaluation artifacts.

Research Content

The research models CareerFit as a Perception-Decision-Action-Learning-Audit workflow. It first identifies recruitment roles, requirements, and control boundaries; then designs and implements the data, matching, recommendation, feedback, automation, email action, and audit modules. Finally, the study evaluates algorithm behavior, software correctness, browser workflows, authorization, runtime health, and local latency within the stated experimental scope.

Key Contributions

The principal contribution of this thesis is the design and implementation of an integrated, controlled recruitment workflow rather than the invention of a new machine-learning model. The contributions are summarized as follows:

- An integrated recruitment platform that combines public job discovery, reviewed CV processing, CV–JD matching, personalized recommendation, application management, Recruiter Jobs, and a Talent Pool workspace.
- A transparent scoring design that keeps the TF-IDF/cosine direct-match score separate from the versioned Potential assessment and explains transferable skills, shared foundations, and seniority checks.
- A feedback-learning workflow based on the Rocchio method, designed to update learned representations from explicit feedback and to support reproducible recomputation rather than uncontrolled cumulative modification.
- A policy-driven AutoFit layer that separates relevance scoring from business actions and applies Human-in-the-Loop controls, user consent, operational constraints, and previous interaction states.
- An actionable email mechanism with expiring, hashed tokens, a non-mutating GET confirmation page, POST redemption, and action-state tracking.
- A traceable evaluation structure that distinguishes algorithm experiments from functional, integration, acceptance, performance, reliability, and security evidence, while explicitly documenting threats to validity.

Thesis objective	Implemented contribution	Evaluation evidence
Analyze IT recruitment workflows	Role-specific requirements and Human-in-the-Loop boundaries	Use cases and traceability matrix
Implement CV–JD matching	TF-IDF, cosine scoring, explanations, and persisted Matchings	Algorithm benchmark and backend tests
Learn from feedback	Rocchio updates followed by scheduled recomputation	Baseline-versus-learned ranking metrics
Support controlled automation	AutoFit policy, quota, cooldown, notification, and audit controls	Security tests and P0 E2E flows
Deliver an integrated platform	Spring Boot, React/TypeScript, PostgreSQL, and Flyway	Build, runtime health, and browser evidence

Table 1.1. Mapping of objectives, contributions, and evidence

Thesis Structure

The report contains an Introduction, four chapters, and a Conclusion. Chapter 1 defines the problem, stakeholders, requirements, and use cases. Chapter 2 presents the theoretical background and solution design. Chapter 3 explains the implementation. Chapter 4 presents the evaluation method, evidence, and threats to validity. The Conclusion discusses achievements, limitations, and future work.

MAIN CONTENT

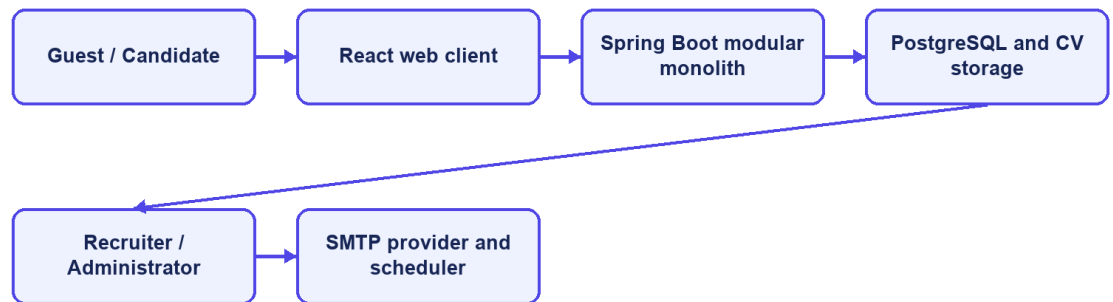
CHAPTER 1. PROBLEM DESCRIPTION AND REQUIREMENTS

1.1 System Analysis Context

CareerFit is designed as a role-based system for IT recruitment. Its boundary includes the React frontend, Spring Boot backend, PostgreSQL database, CV storage, background scheduler, and optional email provider. External job boards, interview scheduling, offers, payroll, and unrestricted automated hiring remain outside the implemented scope.

The central design distinction is between evidence, business state, and action. A matching record contains evidence about the similarity between one CV and one job. An application record represents a candidate's participation in a recruitment process. An automation policy represents consent and operational preferences. These concepts must remain separate: a high similarity score must not silently become an application, and an application status must not be interpreted as a relevance label.

System context



CareerFit IT AutoPilot — thesis diagram

Figure 1.3. CareerFit system context

1.2 Stakeholders and Actors

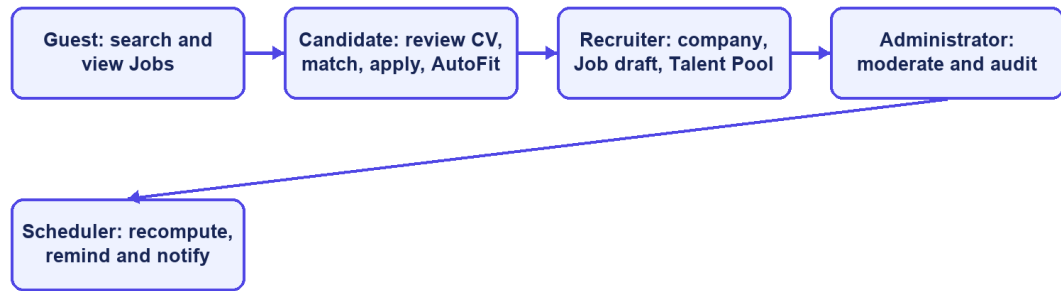
CareerFit serves four interactive roles and two supporting actors. Guest is a frontend state rather than a persisted account role. Candidate, Recruiter, and Administrator are persisted roles in `user_account`. Background processing and the mail provider are non-human actors that execute scheduled work and deliver messages.

Table 1.2. Actors and responsibilities

Actor	Primary responsibilities	Access boundary
Guest	Browse public Jobs, suggestions, details, employers, urgent items, and public market analytics	Public GET operations only; authentication is required for personalized data or applications
Candidate	Maintain profile and portfolio; draft/review/confirm CVs; view matches and recommendations; apply/withdraw; give feedback; configure AutoFit	Candidate-owned resources and Candidate APIs
Recruiter	Maintain company profile and Jobs; review applicants/Talent Pool; bookmark and invite Candidates; update application states	Recruiter APIs and Jobs owned by the authenticated Recruiter
Administrator	Monitor summaries, users, Jobs, email actions, and audit records; moderate Jobs and run controlled maintenance	Administrative APIs only
Background scheduler	Recompute stale Matchings, send digests/reminders, clean expired actions, and run eligible automation	Internal service calls under system control
Mail provider	Deliver notification and signed action emails generated by the backend	SMTP boundary; it does not own CareerFit business state

Stakeholder needs sometimes conflict. Candidates want relevant opportunities, privacy, and control over automatic applications. Recruiters want efficient prioritization without losing access to applicants who do not rank highly. Administrators need visibility and recoverability without unrestricted exposure of CV content. The design therefore combines role authorization, ownership checks, policy settings, validation, explicit statuses, and audit records rather than relying on a single ranking score.

Role-oriented use cases



CareerFit IT AutoPilot — thesis diagram

Figure 1.4. CareerFit use-case overview

1.3 Functional Requirements

The detailed Software Requirements Specification contains individual requirement identifiers. For the thesis, they are consolidated into functional groups that map to implemented modules and observable workflows.

Table 1.3. Consolidated functional requirements

Group	Required capabilities	Main actor
Authentication and account	Registration, email/password login, current-account lookup, role enforcement, and settings	Candidate, Recruiter, Administrator
Public Job portal	Server-side search, suggestions, filters, urgent flag, sorting, pagination, Job details, employers, and public analytics	Guest and authenticated users
Candidate profile and CV	Profile/portfolio, multiple CVs, manual drafts, upload, OCR cleanup, section review/edit/confirm, default CV, and quality signals	Candidate
Job and employer management	Required company profile; JD quality preview; Job draft, publish, update, urgent/deadline fields, status, export, and counts	Recruiter
Matching and recommendation	Direct CV-JD score, labels/reasons, separate Potential assessment, Candidate cards, recruiter ranking, recommendations, and similar Jobs	Candidate and Recruiter

Group	Required capabilities	Main actor
Application and Talent Pool	Apply, history, withdrawal, applicant review, Potential discovery, bookmark, invitation, and status update	Candidate and Recruiter
Feedback learning	Record explicit feedback, update learned Job vectors with Rocchio, and mark Matchings for recomputation	Candidate
Automation and notification	Per-account enable/pause, thresholds, categories, quota, cooldown, quiet hours, run-now, auto-apply, reminders, and delivery logs	Authenticated user and Scheduler
Analytics and administration	Live summaries, event tracking, user/Job moderation, audit logs, notifications, and email-action monitoring	Candidate, Recruiter, Administrator

The implementation exposes these groups through REST-oriented endpoints. Examples include `/api/jobs/search`, `/api/cv/upload`, `/api/cv/{cvId}/review`, `/api/matches/me/cards`, `/api/recommendations/jobs`, `/api/applications`, `/api/recruiter/jobs/{jobId}/applicants`, `/api/recruiter/talent/jobs/{jobId}/bookmarks`, `/api/automation/policy`, and `/api/admin/audit-logs`. Chapter 3 explains the main behavior.

1.4 Non-Functional Requirements

Non-functional requirements constrain how the workflows must operate. They are expressed as design targets; Chapter 4 must distinguish targets from properties verified by measurement.

Table 1.4. Non-functional requirements and design responses

Quality attribute	Requirement	Design response
Security	Protected resources must require authenticated and authorized access	Stateless Spring Security filter chain, JWT processing, role rules, ownership checks, BCrypt password hashing, CORS allow-list, CSP, and HSTS configuration
Privacy	CV and account data must not be public or unnecessarily logged	Candidate-scoped APIs, local protected storage boundary, selective DTOs, and audit metadata rather than full document content
Reliability	Repeated or failed operations should not create inconsistent domain state	Database constraints, transactions, unique keys, optimistic version columns, idempotency checks, explicit statuses, and retry-aware background work

Quality attribute	Requirement	Design response
Performance	Public queries and ranking views should remain responsive for the academic dataset	Database indexes, pagination, stored vectors, background scoring, and bounded result sets
Maintainability	Domain changes should remain localized and database evolution reproducible	Modular package structure, controller–service–repository separation, DTOs, configuration properties, Flyway migrations, and automated tests
Usability	Users must understand loading, errors, empty results, score context, and available actions	Role-specific pages, normalized labels and reasons, validation signals, and explicit action controls
Auditability	Significant automated or administrative events should be reconstructable	audit_log, notification delivery records, email-action state, timestamps, actor/source metadata, and administrative views
Explainability	The system should expose evidence without presenting a score as a hiring probability	Lexical term/skill reasons, separate labels and application states, policy settings, and documented limitations

Security and auditability are defense-in-depth properties. A role rule at the URL layer does not replace ownership verification in a service, and an audit row does not make an unsafe action safe. Similarly, a database index is a performance design decision, not proof of latency under load. The evaluation chapter must test each critical claim in the environment in which it is reported.

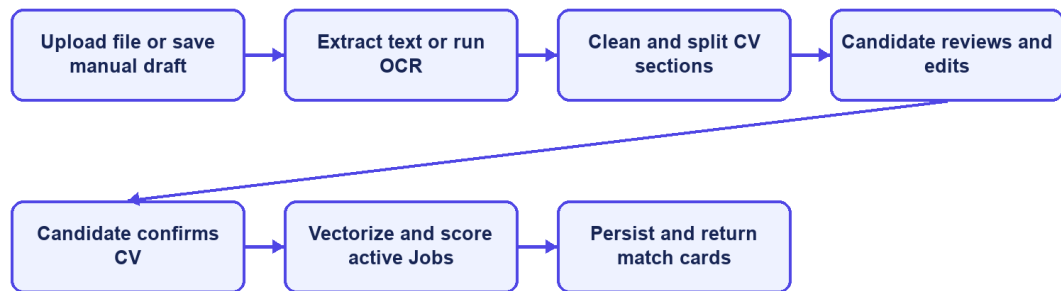
1.5 Use-Case Analysis

1.5.1 Candidate Uploads a CV and Receives Matches

Table 1.5. Use case - Candidate uploads a CV and receives matches

Field	Description
Use-case ID	UC-01
Primary actor	Candidate
Preconditions	The Candidate account is active, authenticated, and has permission to manage its own CVs.
Trigger	The Candidate uploads a supported document or saves manual CV content.
Main flow	Validate and extract the source; preprocess and OCR when needed; create review sections; show warnings; let the Candidate edit and confirm; normalize and vectorize; score active Jobs; persist unique Matchings; return ordered cards.
Alternative/exception flows	Save manual work as DRAFT; reject unsupported or unsafe content; record extraction failure; keep REVIEW_REQUIRED until the Candidate confirms; show an empty state when no Job is eligible.
Postconditions	The CV remains DRAFT/REVIEW_REQUIRED, reaches SCORING_DONE after confirmation, or ends in FAILED with a visible reason.

CV review and matching sequence



CareerFit IT AutoPilot — thesis diagram

Figure 1.5. CV ingestion and matching sequence

1.5.2 Candidate Searches, Applies, and Withdraws

Table 1.6. Use case - Candidate searches, applies, and withdraws

Field	Description
Use-case ID	UC-02
Primary actor	Guest or Candidate
Preconditions	Public search needs no account; application and personalized scoring require an authenticated Candidate.
Trigger	The user searches Jobs, opens details, or selects Apply/Withdraw.
Main flow	Search and filter public Jobs; open Job details; resolve the Candidate and selected/default CV; verify the Job; prevent duplicate Candidate-Job applications; create an application; list owned applications; withdraw when the current state permits.
Alternative/exception flows	Reject missing authentication, missing/default CV, invalid Job, duplicate application, unrelated ownership, or invalid withdrawal state.
Postconditions	A valid application is created or withdrawn, and the resulting state is returned to the owning Candidate.

1.5.3 Recruiter Creates a Job and Reviews Candidates

Table 1.7. Use case - Recruiter creates a Job and reviews candidates

Field	Description
Use-case ID	UC-03
Primary actor	Recruiter
Preconditions	The Recruiter account is active, authenticated, and has completed its company profile.
Trigger	The Recruiter saves a Job draft or opens an owned Job/Talent Pool workspace.
Main flow	Validate structured and free-text data; preview JD quality; save DRAFT; set deadline/urgent state; publish; vectorize after commit; review applicants and Potential CVs; bookmark, invite, or update application state.

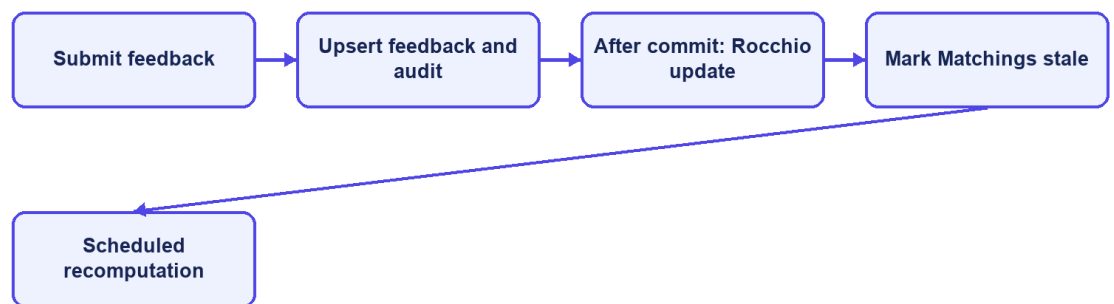
Field	Description
Alternative/exception flows	Reject missing company data, invalid deadline/content, another Recruiter's Job, invalid invitation, or unsupported status transition; keep incomplete work as DRAFT.
Postconditions	The Job draft or published Job and permitted recruitment actions are persisted with ownership and audit evidence.

1.5.4 Candidate Feedback Changes Later Ranking

Table 1.8. Use case - Candidate feedback changes later ranking

Field	Description
Use-case ID	UC-04
Primary actor	Candidate
Preconditions	The Candidate owns the CV and Matching associated with the feedback.
Trigger	The Candidate submits GOOD_MATCH, POTENTIAL, BAD_MATCH, or NOT_INTERESTED feedback.
Main flow	Resolve the Matching and actor; verify ownership and role; upsert one current judgment; commit the transaction; start Rocchio learning after commit; mark affected Matchings for recomputation; rescore and clear the marker.
Alternative/exception flows	Reject another role, another Candidate's Matching, missing evidence, or invalid feedback; retain retry visibility when asynchronous recomputation fails.
Postconditions	The feedback and audit state are stored, and later rankings use the rebuilt learned vector.

Feedback recomputation sequence



CareerFit IT AutoPilot — thesis diagram

Figure 1.6. Feedback learning and recomputation sequence

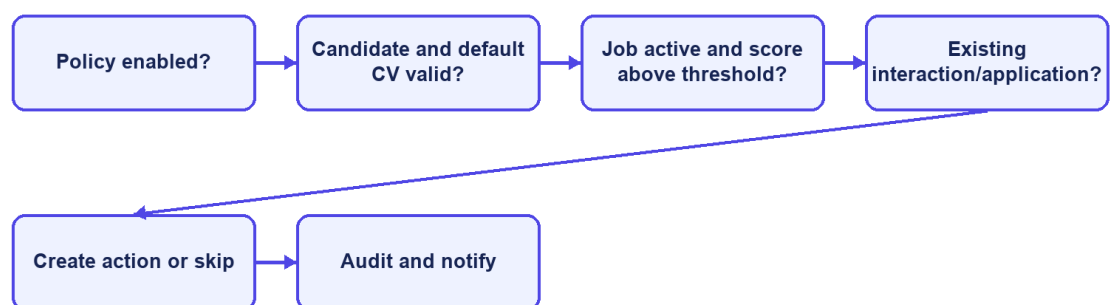
1.5.5 AutoFit Executes a Policy-Governed Action

Table 1.9. Use case - AutoFit executes a policy-governed action

Field	Description
Use-case ID	UC-05
Primary actor	Candidate and background scheduler

Field	Description
Preconditions	An enabled owned policy, an eligible default CV, completed scoring, and an allowed action state exist.
Trigger	A scheduled scan or controlled run-now request evaluates the policy.
Main flow	Resolve the user, Candidate, CV, matches, thresholds, consent, quota, cooldown, quiet hours, time zone, and prior interactions; select eligible actions; create notifications or applications; write audit evidence.
Alternative/exception flows	Skip disabled, paused, ineligible, duplicate, quota-exceeded, cooldown, quiet-hour, stale, or invalid-state items; isolate per-item failures.
Postconditions	Only policy-allowed actions are recorded, and the remaining items stay unchanged for later evaluation.

AutoFit decision flow



CareerFit IT AutoPilot — thesis diagram

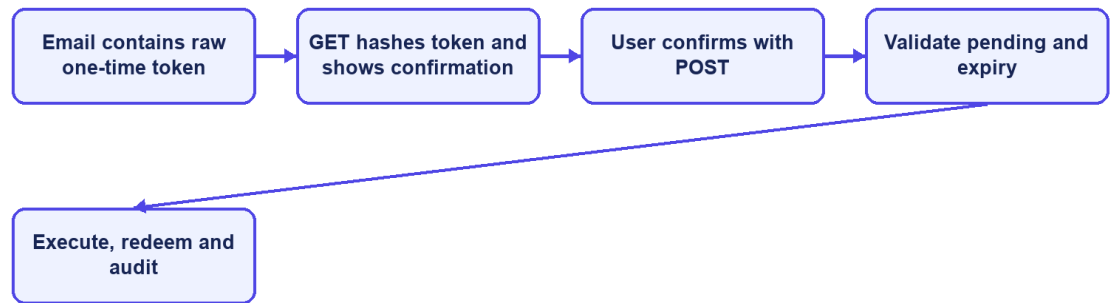
Figure 1.7. AutoFit decision flow

1.5.6 User Completes an Email Feedback Action

Table 1.10. Use case - User completes an email feedback action

Field	Description
Use-case ID	UC-06
Primary actor	Email recipient
Preconditions	A pending, unexpired EmailAction exists and only its SHA-256 token hash is stored.
Trigger	The recipient opens the link and deliberately confirms the action.
Main flow	GET hashes and validates the token and displays a confirmation page without changing recruitment data; POST repeats validation, performs the supported action, marks the token redeemed, and records the outcome.
Alternative/exception flows	Reject missing, expired, redeemed, malformed, or unsupported actions; mark expired pending records; do not execute state changes from GET.
Postconditions	The requested action is executed at most once under normal validation and remains auditable.

Hardened email-action sequence



CareerFit IT AutoPilot — thesis diagram

Figure 1.8. Secure email-action confirmation and redemption sequence

1.6 Chapter Summary

This chapter defined CareerFit's actors, functional and non-functional requirements, and six representative use cases. The requirements separate matching evidence, application state, user policy, automated actions, and audit records so that later design and implementation decisions can be traced to expected behavior.

CHAPTER 2. THEORETICAL BACKGROUND AND SOLUTION DESIGN

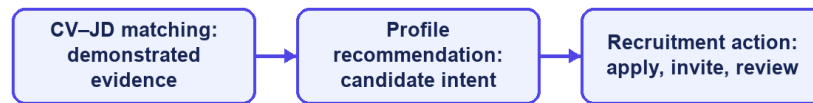
2.1 Recruitment Information and CV–Job Description Matching

Recruitment matching is an information-retrieval and decision-support problem that compares different types of evidence about candidates and vacancies. A curriculum vitae usually describes education, work history, projects, technical skills, certifications, languages, and contact information. A job description lists responsibilities, required and preferred skills, seniority, location, employment conditions, and company context. Some data is structured, such as years of experience or location, while other data is free text and may use inconsistent terms. Therefore, exact database filters are useful for fixed conditions but are not enough to rank the overall fit between a candidate and a position.

The same information supports different user tasks. From the candidate perspective, the system retrieves and ranks jobs that fit a CV or desired profile. From the recruiter perspective, it ranks applicants or discovers potential candidates for a specific vacancy. These directions share document representations and similarity functions, but they are not identical business decisions. A relevant job recommendation does not imply that an application has been submitted, and a high candidate–job similarity does not imply that a recruiter should accept the candidate. The score is therefore evidence for prioritization rather than a final employment decision.

Recruitment data also has domain-specific ambiguity. A technology may be expressed by an abbreviation, a product name, or a broader skill family; job titles vary across companies; and the same term may have different importance at junior and senior levels. Open recruitment corpora are relatively uncommon because CVs contain personal information. The Djinni Recruitment Dataset illustrates both the scale of the matching problem and the value of anonymized, documented corpora, providing more than 150,000 jobs and 230,000 candidate profiles in English and Ukrainian [6]. CareerFit is narrower: it focuses on IT recruitment and uses explicit fields and normalized text so that matching behavior can be inspected within the project scope.

Three distinct recruitment concepts



CareerFit IT AutoPilot — thesis diagram

Figure 2.1. Distinction between matching, recommendation, and recruitment action

2.2 Text Representation and Preprocessing

2.2.1 Document Normalization

Before vectorization, text must be converted into a consistent form. A typical pipeline extracts machine-readable text, normalizes character encoding and letter case, separates or standardizes tokens, removes formatting noise, and may filter stop words or apply stemming or lemmatization. For recruitment documents, preprocessing must keep important technical tokens such as framework names, programming languages, versions, and abbreviations. Removing too much information can make a CV and a JD appear less related even when they share an important technology.

Bilingual data adds further complexity. Vietnamese contains diacritics and multi-syllable expressions separated by spaces, while English technical terminology frequently appears unchanged inside Vietnamese sentences. A practical pipeline must apply deterministic normalization to equivalent inputs and maintain a controlled vocabulary. It should also keep structured attributes—such as location, language, seniority, and salary mode—available for validation or filtering instead of forcing every business constraint into a single text vector.

Preprocessing quality directly affects every later score. Missing OCR text, malformed documents, extremely short descriptions, duplicated boilerplate, and contradictory structured fields can distort term frequencies. For this reason, validation belongs before scoring. Hard validation rejects inputs that cannot support

meaningful processing, while soft validation allows processing but records warnings or suggestions. This distinction prevents the ranking engine from silently assigning apparently precise scores to unusable data.

2.2.2 Bag-of-Words Representation

In a bag-of-words representation, a document is represented by the terms it contains and their weights; word order is not modeled directly. If the vocabulary contains $|V|$ terms, each document is mapped to a vector in $\mathbb{R}^{|V|}$. This representation is sparse because a particular CV or JD contains only a small portion of the complete vocabulary. Its main advantages are computational simplicity, reproducibility, and the ability to inspect which terms contribute to similarity. Its main limitation is lexical dependence: synonyms and related skills do not match unless normalization or an explicit mapping connects them.

2.3 TF-IDF and Cosine Similarity

2.3.1 Term Frequency and Inverse Document Frequency

The vector-space model represents documents and queries as weighted term vectors and ranks documents according to their geometric relationship [1]. TF-IDF is a standard weighting family for this model [2]. Term frequency (TF) reflects how strongly a term occurs in a particular document. Inverse document frequency (IDF) reduces the influence of terms that occur in many documents and increases the relative influence of terms that distinguish a smaller subset of the corpus.

Let $tf(t,d)$ denote the frequency of term t in document d , let N be the number of documents in the reference corpus, and let $df(t)$ be the number of documents containing t . A basic formulation is:

$$tfidf(t,d) = tf(t,d) \times \log(N / df(t)).$$

Implementations often use logarithmic TF scaling, smoothed IDF, or vector normalization to handle repeated terms and unseen values. The exact formulation changes numeric scores, so a thesis must document the implemented formula rather than referring to TF-IDF as if it were a single universal function. A controlled or static reference corpus can improve score stability because the IDF of a term does not change whenever a user adds one document; however, it may become less representative as technologies and hiring terminology evolve.

2.3.2 Cosine Similarity

Cosine similarity compares the direction of two vectors rather than their unnormalized magnitude. For vectors x and y , it is defined as:

$$\cos(x,y) = (x \cdot y) / (||x||_2 ||y||_2).$$